

Ionic + Angular

Guilherme B. Spiger, João Pedro Leite e João Pedro Gaspar



Conteúdo

- 1 Apresentação
- 2 Por que usar?
- 3 Comandos básicos
- 4 Progressive Web App (PWA)
- 5 O que são Cordova e Capacitor?
- 6 Cordova vs. Capacitor
- 7 Estrutura do Projeto
- 8 App demonstração

- O Ionic é um framework de desenvolvimento de aplicativos móveis e web que permite criar aplicações híbridas usando tecnologias web como HTML, CSS e JavaScript. Ele oferece uma série de componentes e ferramentas que facilitam o desenvolvimento de apps que podem ser executados em diferentes plataformas, como iOS, Android e web, a partir de uma única base de código.
- Inicialmente totalmente atrelado ao Angular. Atualmente possui integração com os frameworks:
 - ▶ Angular.
 - ▶ React.
 - ▶ Vue.

Por que usar?

- Implementar uma só base de código front-end e exportar para múltiplas plataformas: (Web, Android e IOS).
- Aproveitar conhecimento prévio em tecnologias web (HTML, CSS, JS).
- Free and Open Source (MIT License). Pode ser usado em projetos pessoais e comerciais gratuitamente.
 - ▶ Possui funcionalidades pagas (Appflow), publicação direta nas app stores, live updates via dashboard SSO e etc.
 - ▶ Código fonte: <https://github.com/ionic-team/ionic-framework>
- Tem componentes prontos que emulam as UI nativas de cada sistema (Android, IOS) para seguir os padrões de cada sistema.
- Lema: "Write once, run anywhere."

- Acesso simplificado as API de cada sistema (abstração):



```
1 import { Injectable } from '@angular/core';
2 import { Storage } from '@ionic/storage-angular';
3
4 @Injectable({
5   providedIn: 'root'
6 })
7 export class AppStorageService {
8
9   constructor(private storage: Storage) {
10     this.init();
11   }
12
13   async init() {
14     await this.storage.create();
15   }
16 }
```

- Utilizando o storage de forma abstrata:



```
1  async saveCompanyData(data: { companyName: string; barberName: string; address: string }): Promise<void> {
2      const { companyName, barberName, address } = data;
3
4      await this.storage.set('companyName', companyName);
5      await this.storage.set('barberName', barberName);
6      await this.storage.set('address', address);
7  }
8
9
10 async loadCompanyData(): Promise<{ companyName: string; barberName: string; address: string }> {
11     const companyName = await this.storage.get('companyName');
12     const barberName = await this.storage.get('barberName');
13     const address = await this.storage.get('address');
14
15     return { companyName, barberName, address };
16 }
```

Comandos básicos

```
1  # Install
2  $ npm install -g @ionic/cli native-run cordova-res
3
4  # Create new app
5  $ ionic start photo-gallery tabs --type=angular --capacitor
6
7  # Run web app
8  $ ionic serve
9
10 # Create a new service class
11 $ ionic g service services/photo
12
13 #Export project
14 $ ionic cap add ios
15 $ ionic cap add android
16
17 #Update changes from web to apps
18 $ ionic cap copy
19 $ ionic cap sync
```

Progressive Web App (PWA)

- Progressive Web Apps (PWAs) são aplicativos da web que utilizam tecnologias modernas para oferecer uma experiência semelhante a aplicativos nativos.
- PWAs funcionam bem em conjunto com o Capacitor, permitindo implantar o aplicativo como um PWA, além de aplicações nativas, aproveitando ambos os canais.

O que são Cordova e Capacitor?

- **Cordova**

- ▶ Plataforma de desenvolvimento de aplicativos móveis híbridos de código aberto.
- ▶ Permite que aplicativos web acessem funcionalidades nativas do dispositivo através de plugins.
- ▶ Usado com o Ionic para construir e implantar aplicativos em iOS, Android e outras plataformas.

- **Capacitor**

- ▶ Plataforma de runtime de código aberto desenvolvida pela equipe do Ionic.
- ▶ Facilita o acesso a APIs nativas e a criação de Progressive Web Apps (PWAs).
- ▶ Alternativa moderna ao Cordova, com melhor integração para aplicativos Ionic.

Cordova vs. Capacitor

- **Capacitor**

- ▶ Moderno e mantido pela equipe do Ionic.
- ▶ Suporte para Web e plataformas nativas.
- ▶ Configuração mínima para integração com APIs nativas.
- ▶ Utiliza APIs modernas e é mais fácil de manter e atualizar.
- ▶ Oferece plugins de código aberto, mas permite usar plugins do Cordova.

- **Cordova**

- ▶ Mais antigo e amplamente utilizado.
- ▶ Base para muitos projetos híbridos legados.
- ▶ Requer configurações adicionais para integração com APIs nativas.
- ▶ Pode ter limitações com APIs modernas e novas funcionalidades.
- ▶ Grande suporte da comunidade e variedade de plugins.

Estrutura do Projeto

- **src/**
 - ▶ **app/**: Contém os módulos e componentes principais do aplicativo.
 - ▶ **assets/**: Recursos estáticos, como imagens, ícones, e fontes.
 - ▶ **environments/**: Arquivos de configuração específicos para diferentes ambientes (produção, desenvolvimento).
 - ▶ **index.html**: Arquivo HTML principal que serve como entrada para o aplicativo.
 - ▶ **main.ts**: Ponto de entrada do aplicativo, responsável por inicializar o Angular.
 - ▶ **theme/**: Contém arquivos de estilo global do projeto (ex: `variables.scss`).
- **ionic.config.json**
 - ▶ Configurações específicas do projeto Ionic, como nome do aplicativo e tipo de projeto.
- **angular.json**
 - ▶ Configurações do Angular CLI, incluindo caminhos de build e configurações de projeto.
- **package.json**
 - ▶ Lista as dependências do projeto, scripts de construção etc.
- **www/**
 - ▶ Diretório onde os arquivos compilados são colocados para distribuição. Gerado durante o processo de build.

- Apresentação de código e funcionalidade do App BarberShop.